Probing the Design Choices of TD-MPC

Vincent Zhou
Master MVA
vincent.zhou@polytechnique.edu

Aziz Bacha
Master MVA
aziz.bacha@polytechnique.edu

Context and Objective

This paper aims to study the TD-MPC framework introduced by Hansen et al. [3], which combines model predictive control (MPC) with temporal-difference (TD) learning in a learned latent space.

TD-MPC aims to learn a sample-efficient controller that can exploit additional computation at test time through planning. It builds on data-driven model predictive control, where actions are selected by planning with a learned dynamics model rather than the true environment. To address the difficulty of long-horizon planning caused by compounding model errors and computational costs, TD-MPC combines short-horizon MPC with temporal-difference learning, augmenting local planning with learned long-term value estimates and control-oriented representations. In figure 1, we provide a high-level diagram of the TD-MPC framework.

We found several strengths to the paper but also some weaknesses, and design choices that are not entirely justified, which opens up possibilities for adaptations.

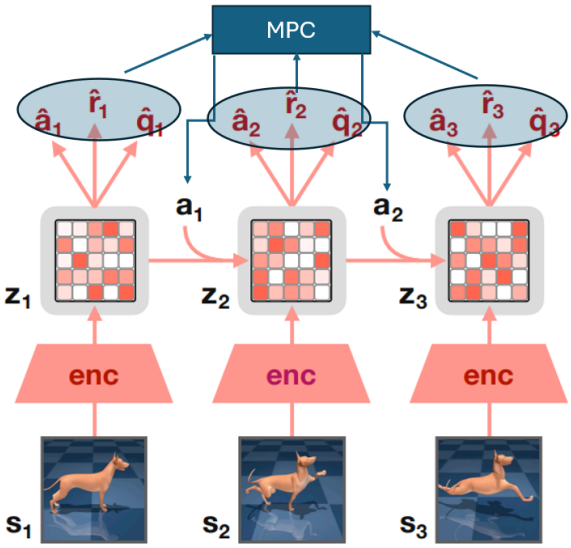

Figure 1: High-level overview of TD-MPC framework. The state s_i is encoded into a latent space z_i which has its own latent dynamics and rewards, used for planning by the MPC framework, which then selects an action a_i

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

1 TD-MPC

1.1 Planning with TD-MPC

TD-MPC performs online planning using model predictive control in a learned latent space. At each decision step, the environment state s_0 is encoded into a latent representation $z_0 = h_\theta(s_0)$. Planning is then carried out over a finite horizon H using the learned Task-Oriented Latent Dynamics (TOLD) model (see Fig. 2).

Starting from z_0, the latent dynamics model is used to roll out candidate trajectories $\{(z_t, a_t)\}_{t=0}^{H}$ in latent space. For each candidate action sequence, the predicted return is estimated as

$$\sum_{t=0}^{H-1} \gamma^t R_\theta(z_t, a_t) \; + \; \gamma^H Q_\theta(z_H, a_H),$$

where R_θ denotes the learned reward model and Q_ϕ the learned action-value function. This formulation allows TD-MPC to combine short-horizon planning with long-term value estimation: explicit model rollouts are limited to a short horizon H, while the terminal value function approximates the remaining future return.

Trajectory optimization is performed using a Model Predictive Path Integral (MPPI) controller. MPPI maintains a distribution over action sequences and iteratively refines it by reweighting sampled trajectories according to their predicted return. Trajectories with higher estimated return receive larger weights, biasing the sampling distribution toward more promising regions of the action space.

In addition to MPPI, TD-MPC uses a learned policy $\pi_\theta(z)$ to guide planning. Rather than sampling action sequences from an uninformed prior, the policy provides action proposals that serve as an initial mean for the sampling distribution. This policy-guided trajectory optimization significantly improves sample efficiency and accelerates convergence of the MPPI procedure. After optimization, only the first action of the selected sequence is executed in the environment, following the standard receding-horizon MPC paradigm. The planning procedure used at inference time is summarized in Algorithm 1.

Algorithm 1 TD-MPC (*inference*)

Require: θ : learned network parameters
μ^0, σ^0 : initial parameters for $\mathcal{N}$
N, N_π : num sample / policy trajectories
s_t, H : current state, rollout horizon

Encode state $z_t \leftarrow h_\theta(s_t)$ *(Assuming TOLD model)*
for each iteration $j = 1 \dots J$ **do**
  Sample N traj. of len. H from $\mathcal{N}(\mu^{j-1}, (\sigma^{j-1})^2 I)$
  Sample N_π traj. of length H using π_θ, d_θ
    ▷ Estimate trajectory returns ϕ_R using $d_\theta, R_\theta, Q_\theta$, ▷ starting from z_t and initially letting $\phi_R = 0$:
  for all $N + N_\pi$ trajectories $(a_t, a_{t+1}, \dots, a_{t+H})$ **do**
    for $t = 0 \dots H-1$ **do**
      $\phi_R \leftarrow \phi_R + \gamma^t R_\theta(z_t, a_t)$ *(Reward)*
      $z_{t+1} \leftarrow d_\theta(z_t, a_t)$ *(Latent transition)*
    end for
    $\phi_R \leftarrow \phi_R + \gamma^H Q_\theta(z_H, a_H)$ *(Terminal value)*
  end for
  Update μ^j, σ^j as the weighted mean and standard deviation of the top-k trajectories, with weights $\propto \exp(\tau\, \phi_R)$
end for
return $a \sim \mathcal{N}(\mu^J, (\sigma^J)^2 I)$

1.2 Training the TOLD Model

TD-MPC learns a **Task-Oriented Latent Dynamics (TOLD)** model that is explicitly optimized for control rather than reconstruction. The latent representation is trained to retain only information that

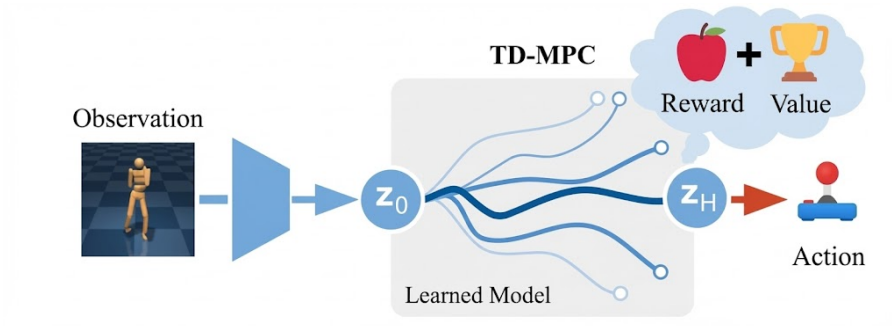

Figure 2: High-level overview of planning with the TD-MPC framework. The state s_i is encoded into a latent space z_i, and then uses the TOLD model to sample trajectories, then uses the best trajectories to form its action and correct itself

is predictive of rewards and values, which is sufficient for planning and policy optimization. All components of the TOLD model are learned jointly using temporal-difference (TD) learning from data collected by the TD-MPC planner.

Training data is stored in a replay buffer $\mathcal{B}$ containing transitions (s_t, a_t, r_t, s_{t+1}) collected during interaction with the environment. During training, trajectory segments of length H are sampled from $\mathcal{B}$ and processed sequentially in latent space. The first observation of each segment is encoded as $z_t = h_\theta(s_t)$, and subsequent latent states are predicted using the learned latent dynamics.

The TOLD model minimizes a discounted objective over the rollout,

$$\sum_{i=t}^{t+H} \lambda^{i-t}\, \mathcal{L}(\theta; \Gamma_i),$$

where $\lambda \in (0, 1]$ downweights errors occurring further in the rollout, and $\mathcal{L}(\theta; \Gamma_i)$ is a per-step loss that decomposes as

$$\begin{aligned} \mathcal{L}(\theta; \Gamma_i) = c_1 \, &\|R_\theta(z_i, a_i) - r_i\|_2^2 \\ &+ c_2 \, \|Q_\theta(z_i, a_i) - (r_i + \gamma Q_{\theta^-}(z_{i+1}, \pi_\theta(z_{i+1})))\|_2^2 \\ &+ c_3 \, \|d_\theta(z_i, a_i) - h_{\theta^-}(s_{i+1})\|_2^2. \end{aligned} \tag{1}$$

The first term trains the reward model to predict immediate rewards from latent states and actions. The second term corresponds to a one-step TD loss for the action-value function, using a target network with parameters θ^-. The third term enforces latent state consistency by aligning the predicted next latent state with the target encoder output of the next observation, thereby stabilizing multi-step rollouts in latent space.

The policy is trained separately from the TOLD model by maximizing the predicted value along latent rollouts. Equivalently, it minimizes the objective

$$\mathcal{J}_\pi(\theta; \Gamma) = -\sum_{i=t}^{t+H} \lambda^{i-t} Q_\theta\big(z_i, \pi_\theta(\mathrm{sg}(z_i))\big),$$

where $\mathrm{sg}(\cdot)$ denotes the stop-gradient operator. This prevents policy updates from backpropagating through the latent representation and helps decouple policy optimization from representation learning.

The complete training procedure, including data collection with TD-MPC, replay buffer updates, TOLD optimization, and target network updates, is summarized in Algorithm 2.

1.3 Strengths and Weaknesses

Strengths.

Algorithm 2 TOLD (training)

Require: θ, θ^{-} : online and target network parameters
Require: η : learning rate, τ : target update rate, λ : rollout discount
Require: $\mathcal{B}$: replay buffer

while training **do**
  // Collect data using TD-MPC
  for $t = 0 \dots T$ **do**
    $a_t \sim \Pi_\theta(\cdot \mid h_\theta(s_t))$
    $(s_{t+1}, r_t) \sim \mathcal{T}(\cdot \mid s_t, a_t),\ \mathcal{R}(\cdot \mid s_t, a_t)$
    $\mathcal{B} \leftarrow \mathcal{B} \cup (s_t, a_t, r_t, s_{t+1})$
  end for
  // Update TOLD using replay buffer
  for number of updates per episode **do**
    Sample trajectory $\{(s_i, a_i, r_i, s_{i+1})\}_{i=t}^{t+H} \sim \mathcal{B}$
    $z_t \leftarrow h_\theta(s_t)$
    $J \leftarrow 0$
    for $i = t \dots t+H$ **do**
      $\hat{r}_i \leftarrow R_\theta(z_i, a_i)$
      $\hat{q}_i \leftarrow Q_\theta(z_i, a_i)$
      $z_{i+1} \leftarrow d_\theta(z_i, a_i)$
      $\hat{a}_i \leftarrow \pi_\theta(z_i)$
      $J \leftarrow J + \lambda^{i-t}\, \mathcal{L}(z_{i+1}, \hat{r}_i, \hat{q}_i, \hat{a}_i)$
    end for
    $\theta \leftarrow \theta - \frac{\eta}{H} \nabla_\theta J$
    $\theta^{-} \leftarrow (1-\tau)\theta^{-} + \tau\theta$
  end for
end while

- **Strong empirical performance**. It shows state-of-the-art performance and sample efficiency across diverse continuous-control benchmarks. As we can see from Fig. 3, it is able to be among the best methods across several tasks.
- **Multimodal RL**. It supports multimodal input thanks to the latent-space planning from high-dimensional observations. The example given in the paper is the Quadruped Corridor setting, where the input is both proprioceptive data and an egocentric camera.

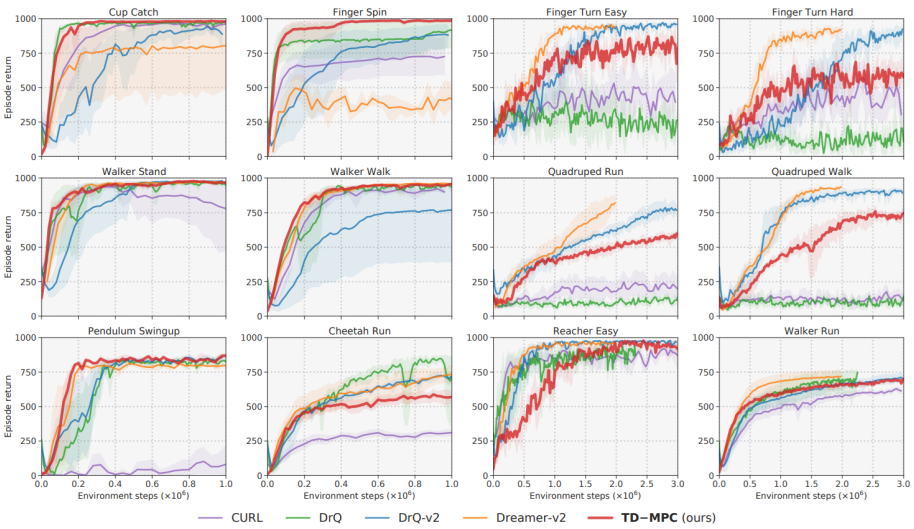

Figure 3: Return of TD-MPC and state-of-the-art algorithms on 12 challenging image-based DMControl tasks (Figure 4 in [3]).

Weaknesses.

- **Single-task training.** TD-MPC is typically trained *per task* (one agent per environment), which limits transfer/generalization and often requires task-specific tuning. Something that will be improved in TDMPC2 ([2])
- **Training/representation stability.** The encoder is trained only through task losses (reward/value/consistency), with no explicit representation objective, which can make the latent dynamics unstable or noise-sensitive. BS-MPC adds a bisimulation-metric loss to regularize the representation and improve robustness [5].
- **Value overestimation from planner–policy mismatch.** Transitions come from the MPC planner, while the policy prior and critic are updated separately; this mismatch can lead to out-of-distribution critic queries and persistent value overestimation. TD-M(PC)^2 mitigates it via a simple policy constraint [4].

2 Our experiments

2.1 Changing the optimizer

The TOLD objective (Eq. 1) is a weighted sum of three losses (reward prediction, TD value loss, and latent consistency). In practice, these terms may have different magnitudes and optimization geometry, which can make training sensitive to the optimizer and its implicit preconditioning.

TD-MPC is trained with Adam in the original paper. Adam uses running averages of first and second moments to form *coordinate-wise* adaptive steps, i.e., it effectively applies a diagonal preconditioner. We therefore evaluate SOAP [6], which introduces *Shampoo matrix preconditioning* and can account for correlations beyond diagonal scaling. SOAP can be viewed as running Adam in the eigenbasis of a Shampoo preconditioner, improving the stability/efficiency of Shampoo-style updates while keeping the procedure relatively simple. [6, 1]

We compare Adam vs. SOAP on the Cartpole Balance task. Fig. 4 shows that SOAP achieves higher training returns than Adam in our setting. In addition, since the TD-MPC paper reports mainly returns and aggregate performance, we also plot the three individual training losses. Surprisingly, none of the loss components decreases monotonically: they remain noisy and can be highly erratic throughout training. SOAP yields slightly smoother / lower losses in parts of training, but the overall behavior remains unstable, suggesting that raw loss curves may be a poor proxy for control performance in this setting.

2.2 n-Step TD learning

In the original TD-MPC paper, the value component of the TOLD loss relies on a *1-step* TD target (see the second term in Eq. 1). Since TD-MPC already unrolls trajectories over a horizon H, it is natural to ask whether using *n-step* TD targets could better exploit multi-step reward information and improve credit assignment. From a classical perspective, increasing n trades bias for variance: longer returns can reduce bootstrapping bias and propagate reward information faster, but may also increase variance and become more sensitive to modeling errors over the rollout.

Concretely, we modify the TD target in Eq. 1 by replacing the 1-step bootstrap with an n-step return. The value loss becomes

$$\left\| Q_\theta(z_i, a_i) - \left(\sum_{k=0}^{n-1} \gamma^k r_{i+k} + \gamma^n Q_{\theta^-}(z_{i+n}, \pi_\theta(z_{i+n})) \right) \right\|_2^2 .$$

We evaluate this modification on the Cartpole Balance task (Fig. 5). In our experiments, n-step TD does not provide a consistent improvement over TD-1: the baseline TD-1 setting already achieves near-optimal returns on this task, leaving limited headroom for gains. We expect the effect of n-step targets to be more visible on harder environments (e.g., locomotion tasks) where reward propagation is more challenging and longer-horizon structure matters more. However, computational constraints restricted our sweep to Cartpole: a single run on Quadruped Run takes on the order of $\sim$1h30 in our setup, whereas Cartpole runs complete in $\sim$10 minutes, enabling faster iteration.

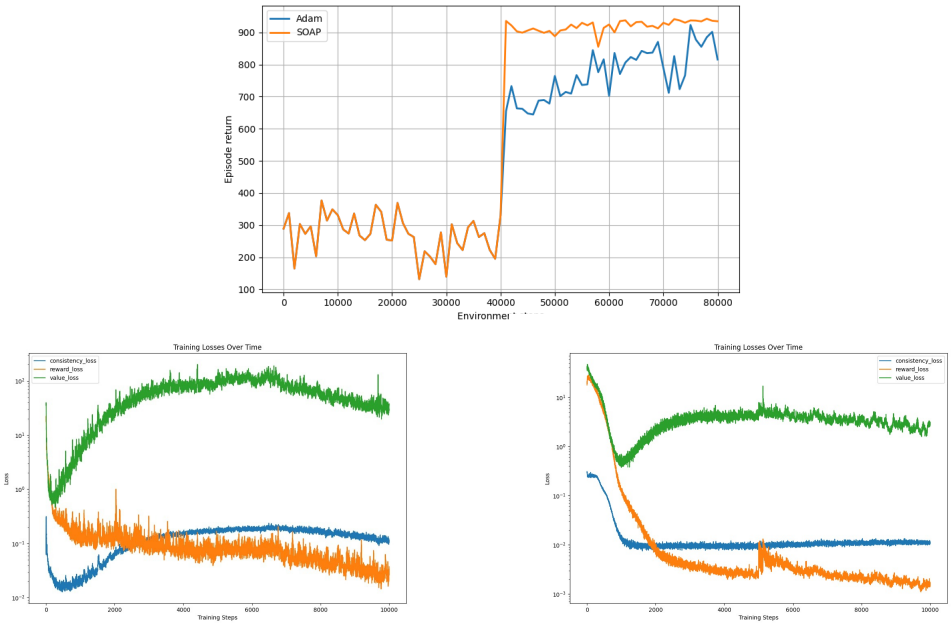

Figure 4: Training return (top) and training loss components with Adam (bottom-left) and SOAP (bottom-right) on Cartpole Balance.

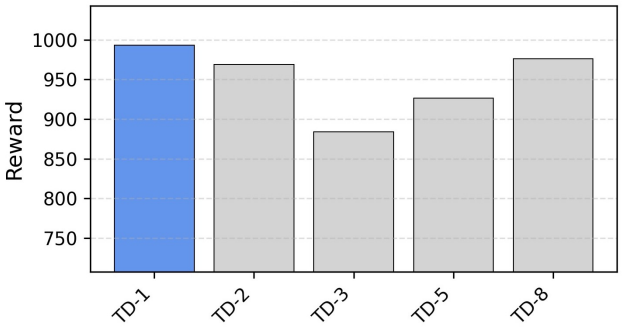

Figure 5: Effect of n-step TD learning on final return in Cartpole Balance.

2.3 Policy update modification

A known difficulty in planner-based model-based RL is that data are generated by the *planner* (policy induced by MPPI), while the nominal policy π_θ is updated with a standard actor objective. Recent work [4] argues that this structural *policy mismatch* can induce persistent value overestimation, since the critic is repeatedly queried on actions that drift away from the planner-induced data distribution. To mitigate this, they propose adding a *policy constraint* that keeps policy updates close to the behavior actions stored in the replay buffer, reducing out-of-distribution action queries during value learning.

Inspired by this idea, we introduce a simple behavior-cloning (BC) prior on replay actions in TD-MPC. Concretely, for a replay trajectory segment Γ we modify the policy objective by penalizing deviation from the stored actions a_i (which correspond to actions executed by the planner during data collection):

$$\tilde{\mathcal{J}}_\pi(\theta;\Gamma) = \sum_{i=t}^{t+H} \lambda^{i-t}\Big[-Q_\theta\big(z_i, \pi_\theta(\mathrm{sg}(z_i))\big) + \alpha_{\text{prior}}\big\|\pi_\theta(\mathrm{sg}(z_i)) - a_i\big\|_2^2\Big].$$

Here, $\alpha_{\text{prior}} > 0$ controls the strength of the prior and $\text{sg}(\cdot)$ denotes stop-gradient as in the original TD-MPC actor update. This modification is conceptually aligned with the constrained/regularized policy improvement viewpoint advocated in [4].

We evaluated this modification on Cartpole Balance and Quadruped Run. In both settings, we did not observe a consistent performance change relative to the baseline. A plausible explanation is that the effect of the prior is sensitive to α_{prior}: if too small, the constraint is negligible; if too large, it can hinder policy improvement by overly anchoring the policy to past planner actions.Additionally, both tasks are already near-solved by the baseline, leaving limited headroom for measurable improvement.

2.4 Adaptative weights

Instead of a fixed (c_1, c_2, c_3) in 1, we tried a simple scheme that rescales them every fixed amount of update proportionally to the current loss magnitudes:

$$c_{\text{reward}} = \frac{\mathcal{L}_{\text{reward}} + \mathcal{L}_{\text{value}} + \mathcal{L}_{\text{cons}}}{\mathcal{L}_{\text{reward}}} \quad \text{and similarly for } c_{\text{value}}, c_{\text{cons}}.$$

Intuitively, terms that are currently larger than other losses get rescaled to be smaller and of the same order of magnitude with other losses.

In practice, this naive rule seems to improve the minimization of the loss, especially for the consistency loss, however we did not see a consistent improvement for the reward function. We can see an example of adding this adaptative weighting for the cartpole balance setting in 6.

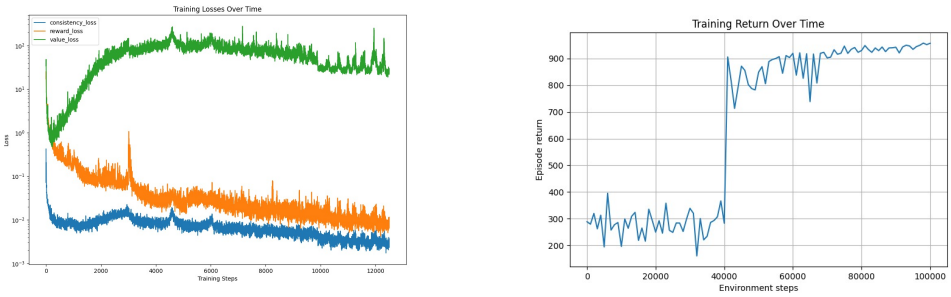

Figure 6: training loss components with adaptative weighting (left) and its training return (right) on Cartpole Balance.

We can also note that the hyperparameter chosen by the authors are already an empirical version of our implementation (doing one test run to see the order of magnitude of each loss then choose the corresponding hyperparameters to rescale them), as they did not simply choose $(1, 1, 1)$ for the choice of the hyperparameters, but chose to scale down the value loss and scale up the consistency loss just like what our implementation would do.

Conclusion

We studied TD-MPC [3], a hybrid model-based RL framework combining latent-space MPC planning with TD learning, and reviewed its main components (planning with MPPI and training the TOLD model). Motivated by several design choices, we ran controlled experiments on a small set of tasks under computational constraints.

Our ablations suggest that optimization and loss design can noticeably affect training dynamics: replacing Adam with SOAP improved training returns on Cartpole in our setup, while loss curves remained noisy and non-monotonic. Extending the critic target to n-step TD did not yield consistent gains on Cartpole, and adding a behavior-cloning prior inspired by TD-M(PC)^2 [4] produced no clear improvement on the tested tasks. Finally, naive adaptive reweighting of loss terms helped reduce imbalance between components but did not consistently translate into higher returns.

Overall, our experiments suggest that TD-MPC is robust and performant, but that its training dynamics can be unstable and sensitive to optimization and loss design, consistent with concerns raised in recent follow-up work on stability and mismatch [5, 4].

Future work could therefore prioritize (1) evaluation on tasks where these failure modes are pronounced (harder locomotion, longer-horizon reward propagation), and (2) reporting mismatch/overestimation diagnostics in addition to returns, to better explain when and why TD-MPC succeeds or fails.

References

[1] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. In *International Conference on Machine Learning*, pages 1842–1850. PMLR, 2018.

[2] Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control, 2024.

[3] Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control. *arXiv preprint arXiv:2203.04955*, 2022.

[4] Haotian Lin, Pengcheng Wang, Jeff Schneider, and Guanya Shi. TD-M(PC)^2: Improving temporal difference mpc through policy constraint. *arXiv preprint arXiv:2502.03550*, 2025.

[5] Yutaka Shimizu and Masayoshi Tomizuka. Bisimulation metric for model predictive control. *arXiv preprint arXiv:2410.04553*, 2024.

[6] Nikhil Vyas, Depen Morwani, Rosie Zhao, Mujin Kwun, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. Soap: Improving and stabilizing shampoo using adam. *arXiv preprint arXiv:2409.11321*, 2024.